Конспект лекции 4

Декодирование по максимуму правдоподобия и решётчатое представление кодов

---

1. Смежные классы линейного кода

Пусть $C \subset \mathbb{F}_2^n$ - линейный двоичный код длины n и размерности k. Тогда $|C| = 2^k$.

Если к каждому кодовому слову $c \in C$ прибавить фиксированный вектор $a \in \mathbb{F}_2^n$, то получим множество

$$C + a = \{c + a \mid c \in C\}.$$

Такое множество называется **смежным классом** (или **косетом**) кода C.

Если $a \in C$, то

$$C + a = C.$$

Это следует из линейности кода: сумма двух кодовых слов снова является кодовым словом. Поэтому прибавление кодового слова лишь переставляет элементы кода.

Таких векторов a ровно 2^k, то есть столько же, сколько кодовых слов.

Если $a \notin C$, то множество $C + a$ не пересекается с C.

Действительно, если бы существовали $c_1, c_2 \in C$ такие, что

$$c_1 = c_2 + a,$$

то тогда

$$a = c_1 + c_2 \in C,$$

что противоречит условию $a \notin C$.

Пусть теперь $a, b \in \mathbb{F}_2^n$. Тогда

$$C + a = C + b$$

тогда и только тогда, когда

$$a + b \in C.$$

Доказательство.

Если $C + a = C + b$, то для некоторого $c_i, c_j \in C$ имеем

$$c_i + a = c_j + b.$$

Отсюда

$$a + b = c_i + c_j \in C.$$

Обратно, если $a + b \in C$, то для любого $c \in C$

$$c + a = (c + a + b) + b,$$

а поскольку $a + b \in C$ и $c \in C$, то $c + a + b \in C$. Значит, любой элемент из $C + a$ принадлежит $C + b$, и аналогично в другую сторону.

Таким образом, пространство $\mathbb{F}_2^n$ разбивается на непересекающиеся смежные классы по коду C.

Число таких классов равно

$$\frac{2^n}{2^k} = 2^{n-k}.$$

Если обозначить $r = n - k$, то получаем 2^r смежных классов, в каждом из которых по 2^k элементов.

---

2. Дискретный канал без памяти и функция правдоподобия

2.1. Модель канала

Пусть канал задаётся:

- множеством входных символов X,
- множеством выходных символов Y,
- переходными вероятностями $P(y \mid x)$.

Канал называется **без памяти**, если вероятности по координатам перемножаются независимо:

$$P(\mathbf{y} \mid \mathbf{x}) = \prod_{i=1}^{n} P(y_i \mid x_i),$$

где $\mathbf{x} = (x_1, \ldots, x_n)$ — переданное слово, а $\mathbf{y} = (y_1, \ldots, y_n)$ — принятое слово.

2.2. Функция правдоподобия

Пусть по каналу передаются кодовые слова из кода

$$C = \{c_1, c_2, \ldots, c_M\}, \qquad M = 2^k.$$

Если принято $\mathbf{y}$, то **функция правдоподобия** для кодового слова c_m имеет вид

$$P(\mathbf{y} \mid c_m) = \prod_{i=1}^{n} P(y_i \mid c_{m,i}).$$

2.3. Декодирование по максимуму правдоподобия

Декодирование по максимуму правдоподобия (ML-декодирование) состоит в выборе кодового слова, для которого вероятность получения наблюдаемого $\mathbf{y}$ максимальна:

$$\hat{c}_{\mathrm{ML}} = \arg\max_{c_m \in C} P(\mathbf{y} \mid c_m).$$

Если все кодовые слова априорно равновероятны, то этот критерий совпадает с MAP-декодированием.

2.4. Логарифм правдоподобия

Поскольку логарифм монотонно возрастает, максимум можно искать по логарифму:

$$\Lambda_m(\mathbf{y}) = \log P(\mathbf{y} \mid c_m).$$

Для канала без памяти получаем

$$\Lambda_m(\mathbf{y}) = \sum_{i=1}^{n} \log P(y_i \mid c_{m,i}).$$

Если обозначить

$$\lambda(y_i, c_{m,i}) = \log P(y_i \mid c_{m,i}),$$

то

$$\Lambda_m(\mathbf{y}) = \sum_{i=1}^{n} \lambda(y_i, c_{m,i}).$$

Следовательно, задача ML-декодирования сводится к максимизации суммы покоординатных вкладов.

---

3. Логарифмическое отношение правдоподобия и вектор надёжности

3.1. Логарифмическое отношение правдоподобия

Теперь рассмотрим **двоичный вход** канала, то есть $x_i \in \{0, 1\}$.

Для каждого принятого символа y определим логарифмическое отношение правдоподобия:

$$L(y) = \log \frac{P(y \mid 1)}{P(y \mid 0)}.$$

Это число показывает, какой символ лучше согласуется с наблюдением y:

- если $L(y) > 0$, наблюдение y скорее говорит в пользу передачи 1;
- если $L(y) < 0$, наблюдение y скорее говорит в пользу передачи 0;
- если $L(y) = 0$, наблюдение одинаково согласуется с обоими символами.

3.2. Вектор надёжности

Для принятого вектора

$$\mathbf{y} = (y_1, y_2, \ldots, y_n)$$

составляем **вектор надёжности**

$$L(\mathbf{y}) = \big(L(y_1), L(y_2), \ldots, L(y_n)\big).$$

3.3. Отображение кодового слова в $\{\pm 1\}^n$

Для кодового слова

$$c_m = (c_{m,1}, \dots, c_{m,n})$$

введём вектор

$$\xi_m = (\xi_{m,1}, \dots, \xi_{m,n}), \qquad \xi_{m,i} = 2c_{m,i} - 1.$$

То есть

- если $c_{m,i} = 0$, то $\xi_{m,i} = -1$;
- если $c_{m,i} = 1$, то $\xi_{m,i} = +1$.

Это стандартное преобразование из двоичного алфавита $\{0, 1\}$ в симметричный алфавит $\{-1, +1\}$.

---

4. ML-декодирование через скалярное произведение

4.1. Формулировка

Для двоичного входного канала без памяти ML-декодирование эквивалентно поиску кодового слова c_m, для которого максимально скалярное произведение

$$\langle \xi_m, L(\mathbf{y}) \rangle = \sum_{i=1}^{n} \xi_{m,i}\, L(y_i).$$

Иными словами, нужно выбрать кодовое слово, наиболее согласованное с вектором надёжности.

4.2. Идея доказательства

Рассмотрим два кодовых слова c_m и $c_{m'}$ и разобьём множество координат $\{1, \dots, n\}$ на четыре подмножества:

$$I(0,0) = \{i \mid c_{m,i} = 0,\ c_{m',i} = 0\},$$
$$I(1,1) = \{i \mid c_{m,i} = 1,\ c_{m',i} = 1\},$$
$$I(1,0) = \{i \mid c_{m,i} = 1,\ c_{m',i} = 0\},$$
$$I(0,1) = \{i \mid c_{m,i} = 0,\ c_{m',i} = 1\}.$$

Разность скалярных произведений равна

$$\langle \xi_m, L(\mathbf{y}) \rangle - \langle \xi_{m'}, L(\mathbf{y}) \rangle.$$

На множествах $I(0,0)$ и $I(1,1)$ соответствующие слагаемые совпадают, поэтому они сокращаются. Остаётся

$$2\sum_{i\in I(1,0)} L(y_i) - 2\sum_{i\in I(0,1)} L(y_i).$$

Подставляя определение

$$L(y_i) = \log\frac{P(y_i \mid 1)}{P(y_i \mid 0)},$$

и группируя логарифмы, получаем

$$\langle \xi_m, L(\mathbf{y})\rangle - \langle \xi_{m'}, L(\mathbf{y})\rangle = 2\big(\Lambda_m(\mathbf{y}) - \Lambda_{m'}(\mathbf{y})\big).$$

Следовательно, порядок кодовых слов по величине $\langle \xi_m, L(\mathbf{y})\rangle$ совпадает с порядком по логарифму правдоподобия. Значит, максимизация скалярного произведения эквивалентна ML-декодированию.

---

5. Частный случай: ДСК и расстояние Хэмминга

5.1. Логарифмическое отношение правдоподобия в ДСК

Рассмотрим двоичный симметричный канал (ДСК) с вероятностью ошибки p, где

$$0 < p < \frac{1}{2}.$$

Тогда

$$P(y \mid x) = \begin{cases} 1-p, & y = x, \\ p, & y \neq x. \end{cases}$$

Поэтому

$$L(y_i) = \begin{cases} \log\dfrac{1-p}{p}, & y_i = 1, \\ -\log\dfrac{1-p}{p}, & y_i = 0. \end{cases}$$

Обозначим

$$\delta = \log\frac{1-p}{p} > 0.$$

Тогда

$$L(y_i) = \begin{cases} \delta, & y_i = 1, \\ -\delta, & y_i = 0. \end{cases}$$

5.2. Связь со расстоянием Хэмминга

Подставим это в скалярное произведение. Каждый совпадающий символ $c_{m,i} = y_i$ даёт вклад $+\delta$, а каждый несовпадающий - вклад $-\delta$.

Если $d(\mathbf{y}, c_m)$ - расстояние Хэмминга между $\mathbf{y}$ и c_m, то число совпадающих координат равно $n - d(\mathbf{y}, c_m)$, а число несовпадающих - $d(\mathbf{y}, c_m)$. Поэтому

$$\langle \xi_m, L(\mathbf{y}) \rangle = \delta\big((n - d(\mathbf{y}, c_m)) - d(\mathbf{y}, c_m)\big) = \delta\big(n - 2d(\mathbf{y}, c_m)\big).$$

5.3. Вывод

Так как $\delta > 0$, то

$$\langle \xi_m, L(\mathbf{y}) \rangle \to \max \quad \Longleftrightarrow \quad d(\mathbf{y}, c_m) \to \min.$$

Следовательно, в ДСК ML-декодирование эквивалентно **декодированию по минимуму расстояния Хэмминга**.

Это важный вывод: расстояние Хэмминга здесь не просто удобная метрика, а именно оптимальный критерий декодирования.

---

6. Решётчатое представление кодов

6.1. Что такое решётка

Решётка (trellis) — это специальный ориентированный граф, используемый для компактного представления множества кодовых слов.

Для блокового кода длины n решётка обычно имеет $n + 1$ ярусов, пронумерованных от 0 до n.

Основные свойства:

1. вершины разбиты на непересекающиеся ярусы;
2. нулевой и последний ярусы содержат по одной вершине;
3. рёбра идут только между соседними ярусами;

4. каждому ребру приписана метка — соответствующий кодовый символ;
5. при декодировании каждому ребру дополнительно сопоставляется **ветвевой метрический вес**, зависящий от принятого символа.

Путь от начальной вершины до конечной однозначно задаёт кодовое слово.

6.2. Профиль решётки

Профиль решётки - это последовательность чисел вершин на каждом ярусе.

Если на ярусе i находится s_i вершин, то профиль имеет вид

$$(s_0, s_1, \ldots, s_n).$$

Чем меньше профиль, тем проще декодирование: на каждом шаге нужно сравнивать меньшее число путей.

6.3. Пример: код $(3, 2)$

Рассмотрим код с порождающей матрицей

$$G = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{pmatrix}.$$

Его кодовые слова:

$$C = \{000,\ 011,\ 101,\ 110\}.$$

Для этого кода можно построить решётку из 4 ярусов: $0, 1, 2, 3$.

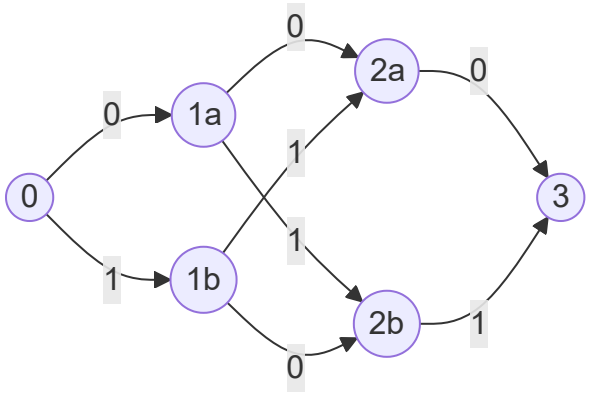
Профиль минимальной решётки:

$$(1, 2, 2, 1).$$

Смысл ярусов:

- ярус 0: начальная вершина;
- ярус 1: две возможности для первого символа;
- ярус 2: два состояния после объединения путей с одинаковым будущим;
- ярус 3: конечная вершина.

6.4. Структура решётки для кода $(3, 2)$



Здесь метки на рёбрах - это символы кодового слова. Каждый путь слева направо даёт одно кодовое слово:

- $0 \to 0 \to 0$, то есть 000;
- $0 \to 1 \to 1$, то есть 011;
- $1 \to 0 \to 1$, то есть 101;
- $1 \to 1 \to 0$, то есть 110.

Ключевой момент состоит в том, что на промежуточных ярусах можно **объединять** пути, если их дальнейшие продолжения совпадают. Именно это уменьшает число состояний.

---

7. Декодирование по решётке на примере

7.1. Постановка задачи

Пусть для кода

$$C = \{000,\ 011,\ 101,\ 110\}$$

из канала принято слово

$$\mathbf{y} = (1, 0, 0).$$

Для ДСК естественный путь декодирования по решётке состоит в накоплении расстояния Хэмминга вдоль пути.

7.2. Первый шаг: переход с яруса 0 на ярус 1

Первый принятый символ равен $y_1 = 1$.

- Переход по ребру с меткой 0 даёт вклад $d(1,0) = 1$.
- Переход по ребру с меткой 1 даёт вклад $d(1,1) = 0$.

Итак, после первого шага метрики равны:

- верхний путь: 1;
- нижний путь: 0.

7.3. Второй шаг: переход с яруса 1 на ярус 2

Теперь $y_2 = 0$.

Из состояния, достигнутого по первому символу 0:

- ребро 0: новая метрика $1 + d(0,0) = 1$;
- ребро 1: новая метрика $1 + d(0,1) = 2$.

Из состояния, достигнутого по первому символу 1:

- ребро 1: новая метрика $0 + d(0,1) = 1$;
- ребро 0: новая метрика $0 + d(0,0) = 0$.

Теперь сравниваем пути, приходящие в одинаковые вершины яруса 2:

- в одно состояние приходят две метрики 1 и 1 - возникает ничья;
- в другое состояние приходят метрики 2 и 0 - сохраняется путь с метрикой 0.

На этом шаге путь, соответствующий слову 011, уже становится хуже конкурентов.

7.4. Третий шаг: переход к финалу

Для $y_3 = 0$:

- из состояния с последним символом 0 добавляется $d(0,0) = 0$;
- из состояния с последним символом 1 добавляется $d(0,1) = 1$.

В итоге минимальная суммарная метрика оказывается равной 1.

7.5. Результат декодирования

Расстояние Хэмминга от $\mathbf{y} = 100$ до кодовых слов равно:

Кодовое слово	**Расстояние до** 100
000	1
011	3
101	1
110	1

Следовательно, ML-декодирование в ДСК даёт не единственный ответ: минимальное расстояние равно 1, и его достигают три кодовых слова

$$000, \quad 101, \quad 110.$$

Если декодер реализован по принципу Витерби и при равенстве метрик выбирает одного из кандидатов произвольно, то он может вернуть любой из этих трёх вариантов.

---

8. Как строится решётка для произвольного кода

8.1. Разбиение кодового слова на прошлое и будущее

Пусть C - код длины n. Для каждого яруса $i = 0, 1, \ldots, n$ любое кодовое слово c разбивается на две части:

$$c = (c^P, c^F),$$

где:

- c^P - **прошлое** (past), то есть первые i символов;
- c^F - **будущее** (future), то есть оставшиеся $n - i$ символов.

8.2. Множество возможных продолжений

Для фиксированного прошлого c^P определим множество всех допустимых продолжений:

$$F_i(c^P) = \{\, c^F \mid (c^P, c^F) \in C \,\}.$$

Это множество показывает, какие хвосты могут следовать после данного префикса.

8.3. Правило объединения вершин

Два узла на ярусе i можно объединить, если их множества будущих продолжений совпадают:

$$F_i(c_1^P) = F_i(c_2^P).$$

Именно это правило даёт минимизацию решётки: разные префиксы с одинаковым будущим поведением нет смысла хранить отдельно.

8.4. Правило построения рёбер

Вершина на ярусе i соединяется с вершиной на ярусе $i+1$, если существует кодовое слово, у которого:

- первые i символов соответствуют первой вершине,
- первые $i+1$ символов соответствуют второй вершине,
- а символ на ребре равен $(i+1)$-й координате этого кодового слова.

Так строится послойный граф, в котором каждый полный путь соответствует ровно одному кодовому слову.

8.5. Таблица для построения

На практике для каждого яруса удобно вести такую таблицу:

Столбец	**Смысл**
Номер яруса	Текущий уровень i
c^P	Все возможные прошлые префиксы длины i
$F_i(c^P)$	Допустимые будущие продолжения
Предыдущие вершины	Из каких узлов пришли пути
Вершина	Номер состояния на текущем ярусе
Кодовый символ	Метка ребра

Такая таблица помогает вручную строить решётку и видеть, где именно возникает схлопывание состояний.

8.6. Пример общего роста числа узлов

Если код имеет параметры $(6, 3)$, то всего в нём $2^3 = 8$ кодовых слов длины 6, а решётка имеет 7 ярусов.

Типичная логика выглядит так:

- на ярусе 0 одна вершина;
- на ярусе 1 обычно два состояния;
- на ярусе 2 возможно до четырёх состояний;
- далее число состояний может как расти, так и уменьшаться за счёт совпадения множеств будущих продолжений.

Именно поэтому профиль решётки зависит не только от самого кода, но и от того, в каком порядке рассматриваются координаты и насколько удачно выполнено объединение состояний.

---

9. Сложность декодирования по решётке

Сложность решёточного декодирования определяется прежде всего профилем решётки.

Если на каком-то ярусе очень много вершин, то на этом шаге приходится сравнивать много частичных путей. Если же число вершин мало, декодирование оказывается значительно дешевле, чем полный перебор всех кодовых слов.

Главное преимущество решётки состоит в следующем:

- на каждом ярусе можно сравнивать пути, приходящие в одно и то же состояние;
- среди них достаточно оставить только путь с наилучшей накопленной метрикой;
- остальные пути можно отбросить, так как они уже не смогут стать оптимальными.

Это и есть основная идея алгоритма Витерби.